



1. Python: Basic Syntaxes

"print(), input(), variables, operators, and more"

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 0. Python: High-Level Interpreted Programming Language](#)

Contents:

[1. The `print\(\)` Statement](#)

[2. Variables](#)

[3. The `input\(\)` Function](#)

[4. Common Operators](#)

[Summary](#)

[References](#)

Every programming language has its own way of writing instructions. Some use symbols everywhere. Others feel extremely strict about structure. Python is different.

Python was designed to be readable first. The goal is not just to make programs work, but to make them understandable. Because of that, Python code often feels closer to human language than traditional programming syntax.

And before we start building larger programs, we need to understand the foundation of that language—the syntax. The small rules that control how Python reads, interprets, and executes our code.

This section is where we finally begin speaking Python properly. You'll learn how Python displays information, receives user input, stores data inside variables, calculates numbers, handles text and special characters, and much

more. Some of these ideas may seem small at first, but together, they form the grammar behind every Python program you will ever write.

So think of this chapter as learning the alphabet before writing full stories. Simple syntax becomes logic. Logic becomes programs. And programs become systems.

1. The `print()` Statement

If Python had a “voice”, then `print()` would be it.

The `print()` statement is one of the most important built-in functions in Python because it allows the program to communicate with us. Whenever we want to display information, test our code, debug values, or simply show a message to the user, we usually use `print()`.

In simple terms, `print()` sends data to the output screen.

And here’s something important to understand early. Python does not only print text. It can also print numbers, equations, variables, results from calculations, multiple objects at the same time. Almost everything can be displayed using `print()`.

- **Basic Printing:**

```
print("Hello World")
print("Hello World", "Hello Python")
```

Hello World

Hello World Hello Python

Notice something interesting here. When multiple objects are placed inside `print()`, Python automatically separates them with a space.

- **Printing Multiple Messages:**

```
# Display three messages
print("Welcome to Python")
print("Python is fun")
print("Problem Driven")
```

Welcome to Python
Python is fun
Problem Driven

Each `print()` statement moves to a new line automatically.

- **Printing Numbers and Equations:** Python can also print numerical values directly.

```
# Print a number or equations
print(3.14)
print("3.5 * 4 / 2 - 2.5 is")
print(3.5 * 4 / 2 - 2.5)
```

3.14
3.5 * 4 / 2 - 2.5 is
4.5

Now here's an important question: Why does this line `print("3.5 * 4 / 2 - 2.5 is")` return a string? Because anything wrapped inside quotation marks (`" "` or `' '`) becomes a string in Python. So Python treats it as plain text—not as a mathematical equation.

But `print(3.5 * 4 / 2 - 2.5)` contains no quotation marks. So Python interprets it as an actual mathematical expression and calculates the result before printing it.

- **Using `sep=` (Separator):** Sometimes we want to control how multiple objects are separated. By default, Python inserts a space between objects. But with the `sep` parameter, we can customize it.

```
# Separator
print("Hello", "how are you?", sep="---")
```

Hello---how are you?

Instead of a space, Python now inserts `"---"` between the two objects.

- Using `end=` : Normally, every `print()` statement ends with a newline character (`\n`). That's why each output appears on a separate line. But we can change this behavior using `end`.

```
print("Hello Python", end="3")
```

Hello Python3

Python does not move to the next line anymore. Instead, it ends the statement with `"3"`.

- **Printing on a Single Line:**

```
# Single line with end
print("Hello", end=" ")
print("Python", end=" ")
print("Programming", end=" ")
```

Hello Python Programming

Instead of jumping to a new line after every `print()`, Python keeps printing on the same line because we replaced the default newline with a space.

At first glance, `print()` may seem simple. But in reality, it becomes one of the most frequently used tools in programming. You'll use it to display results, debug programs, inspect variables, track logic flow, interact with users. In many ways, `print()` is the first bridge between your code and the outside world.

2. Variables

Programs do not remember things magically. Whenever Python needs to store information—names, numbers, calculations, or even entire datasets—it uses something called a variable.

A variable is simply a named container that stores a value. Think of it like attaching a label onto data so the program can reference it later.

```
name = "Alice"  
age = 25  
city = "New York"
```

Here:

- `name` stores a string
- `age` stores an integer
- `city` stores another string

And once stored, we can reuse them anywhere in the program.

```
# Displaying multiple items  
print(name, age, city)
```

Alice 25 New York

Variable Naming Rules - Python allows flexible variable names, but there are important rules.

A variable:

- Cannot start with a number
- Should not contain spaces
- Should not use Python keywords
- Should NEVER use built-in function names like `print`, `input`, `list`, or `type`

Good examples:

```
x = 10
_speed = 50
user_name = "Michael"
```

Bad examples:

```
1x = 10
my name = "Alice"
print = 5
```

Even if Python technically allows overriding built-in functions, doing so is extremely dangerous and confusing.

Variables in Calculations: Variables become truly useful when combined with mathematical operations.

```
# Create variables then use them for calculation
x = 10
y = 20

print("x + y =", x + y)
print("x - y =", x - y)
print("x * y =", x * y)
print("x / y =", x / y)
print("x // y =", x // y)
```

```
x + y = 30
x - y = -10
x * y = 200
x / y = 0.5
x // y = 0
```

Notice the difference:

- `/` performs normal division
- `//` performs floor division (removes decimal values)

Real-World Example

```
top_speed = 160
distance = 300

# Display the values referenced by the variables
print('The top speed is', top_speed)
print('The distance traveled is', distance)
print('The estimated time to reach the distance is', distance / top_speed)
```

The top speed is 160

The distance traveled is 300

The estimated time to reach the distance is 1.875

This is where programming starts becoming practical. We store values, then use them to solve problems dynamically.

- **Multiple Variable Assignment:** Python allows multiple variables to be assigned in a single line. This is something many programming languages (like C) do not support directly.

```
# Area of a rectangle
length, width = 18, 15

area = length * width
circumference = 2 * (length + width)

print("The area of the rectangle is", area)
print("The circumference of the rectangle is", circumference)
```

The area of the rectangle is 270

The circumference of the rectangle is 66

Python focuses heavily on readability and simplicity. And multiple assignment is one of those small features that makes code feel cleaner.

3. The `input()` Function

If `print()` is Python speaking. Then `input()` is Python learning how to ask questions. The `input()` function allows the user to enter information while the program is running.

Whenever Python reaches an `input()` statement, the program pauses, waits for the user, reads the entered value, then continues execution. This is one of the first steps toward interactive programming.

```
print(input("Enter something: "))
```

Enter something: Python
Python

Using `input()` with Variables: Usually, we store user input inside variables.

```
string = input("Enter a String: ")  
print(string)
```

Enter a String: Michael
Michael

Greeting Example:

```
name = input("Enter your name: ")  
print("Hello", name, "!")
```

Enter your name: Alice
Hello Alice !

Now the program feels responsive. It reacts to the user's input dynamically.

Data Types and `input()`

Here's something extremely important: `input()` always returns a string by default. Even if the user types a number. For example:

```
age = input("Enter age: ")
print(type(age))
```

```
<class 'str'>
```

That means we often convert inputs into other data types manually.

Converting Input Types

```
# Add a data type before the input()
name = str(input("Enter Name: "))
age = int(input("Enter Age: "))
salary = float(input("Enter Salary: "))

print(name, age, salary)
```

```
Enter Name: Alice
Enter Age: 25
Enter Salary: 46000
Alice 25 46000.0
```

Here:

- `str()` converts to string
- `int()` converts to integer
- `float()` converts to decimal numbers

Input with Calculation

```
top_speed = float(input("Enter the top speed: "))
distance = float(input("Enter the distance: "))

# Calculation
print('The top speed is', top_speed)
print('The distance traveled is', distance)
print('The estimated time to reach the distance is', distance / top_speed)
```

```
Enter the top speed: 120
Enter the distance: 300
The top speed is 120.0
The distance traveled is 300.0
The estimated time to reach the distance is 2.5
```

And this is the moment where programs become interactive systems instead of static code.

4. Common Operators

Programming would be meaningless without operations. Operators are symbols that allow Python to perform calculations and manipulate values.

You've already seen some of them earlier: `+`, `-`, `*`, `/`. But Python supports many more.

Arithmetic Operators

```

a = float(input("Enter a: "))
b = float(input("Enter b: "))

print("a + b:", a + b)    # Addition
print("a - b:", a - b)    # Subtraction
print("a * b:", a * b)    # Multiplication
print("a / b:", a / b)    # Division
print("a // b:", a // b)  # Floor Division
print("a ** b:", a ** b)  # Exponentiation
print("a % b:", a % b)    # Remainder

```

```

Enter a: 10
Enter b: 3
a + b: 13.0
a - b: 7.0
a * b: 30.0
a / b: 3.3333333333333335
a // b: 3.0
a ** b: 1000.0
a % b: 1.0

```

Understanding the Operators

Operator	Meaning	Example
+	Addition	5 + 2 = 7
-	Subtraction	5 - 2 = 3
*	Multiplication	5 * 2 = 10
/	Division	5 / 2 = 2.5
//	Floor Division	5 // 2 = 2
**	Exponentiation	5 ** 2 = 25
%	Modulus / Remainder	5 % 2 = 1

These operators form the mathematical backbone of Python. Almost every program—from game engines to Machine Learning systems—depends on them in some form. Simple calculations today, eventually become algorithms tomorrow.

Summary

This section introduced the fundamental syntax of Python—the basic grammar behind every Python program.

We started with the `print()` statement, the primary way Python communicates with the outside world. We learned how to display text, numbers, equations, multiple values, and how to control output formatting using parameters like `sep` and `end`.

Then, we explored variables, which allow programs to store and reuse information. Variables became our way of giving names to data, making calculations and logic reusable instead of hardcoded. We also learned important naming rules and saw how Python supports multiple variable assignments in a clean and readable way.

After that, we moved into the `input()` function, where programs stop being static and become interactive. Instead of only displaying information, Python could now receive data from users, store it inside variables, and use it in calculations. We also learned an important detail: `input()` returns strings by default, which is why type conversion using `int()`, `float()`, and `str()` becomes necessary.

Finally, we covered common arithmetic operators such as addition, subtraction, multiplication, division, floor division, exponentiation, and remainder operations. These operators form the computational foundation behind mathematical logic in Python.

At first glance, these concepts may feel small:

- Printing text
- Storing values
- Reading user input
- Performing calculations

But these are the exact building blocks behind larger systems.

Every game, AI model, web application, simulation, or automation tool eventually traces back to these same foundations.

And now... we move one step further.

Because writing code that works is only part of programming.

The next step is learning how to make output cleaner, more readable, and professionally structured.

References

<https://www.geeksforgeeks.org/python/python-output-using-print-function/>

<https://www.geeksforgeeks.org/python/python-input-function/>

<https://www.geeksforgeeks.org/python/python-variables/>

<https://www.geeksforgeeks.org/python/python-arithmetic-operators/>

Next Section:

 [2. Python: Formatting](#)